

PDF Tool — Technical Documentation

Stack, services and architecture of the live web application

Status: **LIVE** · Generated 14 Jul 2026

1. Live addresses

Component	URL / location
Web app (public)	https://pdftool-web.onrender.com
API	https://pdftool-api-xyd5.onrender.com
Source code	github.com/mukeshroyhub/pdf_tool

2. What the app does

An online PDF workspace: users sign in, upload PDFs and images, and run tools in the browser — merge, split, organize pages, edit/annotate, add watermarks, compress, convert PDF ↔ images, redact, and fill forms. Each user has a private file workspace with a storage quota and an activity history. Sign-in supports email/password, Google, and a one-tap guest mode; email verification and password reset work over real email.

3. Architecture

The browser talks to a single origin — the Next.js web app — which transparently proxies all `/api/*` calls to the Express API. The API stores metadata in PostgreSQL (Neon) and streams file bytes to object storage (Supabase), so uploads survive server restarts. Email is sent through Brevo's HTTPS API. The whole app is packaged as Docker images and deployed on Render.

Flow: Browser → Next.js web (single origin) → `/api/*` proxied → Express API → Neon (data) + Supabase (files); email → Brevo API; login → JWT + rotating refresh tokens / Google OAuth.

4. Hosting & external services

Service	Role	Plan
Render	Runs the two Docker services (web + API)	Free
Neon	PostgreSQL database (users, files metadata, activity)	Free (no expiry)
Supabase	S3-compatible object storage — bucket 'pdfforge-files'	Free
Brevo	Transactional email (verification, password reset) via HTTPS API	Free (300/day)
Google Cloud	OAuth 2.0 for 'Continue with Google'	Free
UptimeRobot	Pings the API health check so it never idles to sleep	Free
GitHub Actions	CI: typecheck, lint, tests and build on every push	Free

5. Technology stack

- **Frontend** — Next.js 15, React 19, TypeScript, Tailwind CSS, Radix/shadcn-style UI, TanStack Query, react-hook-form, next-themes (dark mode), pdf.js, lucide-react, sonner, framer-motion.
- **Backend** — Node.js 22, Express, TypeScript, Zod (shared validation between client and server).
- **PDF engine** — pdf-lib, pdfjs-dist, @napi-rs/canvas (page rendering), sharp (images).
- **Data** — Prisma ORM on Neon PostgreSQL (SQLite in local dev).

- **Storage** — @aws-sdk/client-s3 against Supabase; local disk in dev (one swappable driver).
- **Auth** — jsonwebtoken, bcryptjs, google-auth-library; httpOnly rotating refresh-token cookies.
- **Delivery** — Docker; Render for hosting; GitHub Actions for CI.

6. Configuration (environment variables)

Set on the Render API service; secret values live only in the dashboard, never in the code.

Variable	Purpose
DATABASE_URL	Neon PostgreSQL connection string
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	Sign & verify auth tokens
WEB_URL / API_URL	Public URLs (single-origin proxy + OAuth callback)
STORAGE_DRIVER=s3, S3_*	Supabase object storage credentials & endpoint
BREVO_API_KEY, MAIL_FROM	Transactional email via Brevo
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	Google OAuth sign-in

7. Deployment & updates

- Push to the 'main' branch on GitHub → GitHub Actions runs the checks → Render automatically rebuilds and redeploys both services.
- Database and files are external (Neon + Supabase), so the Render instances are stateless and can be rebuilt any time.
- Free-tier note: services would sleep after 15 min idle; an UptimeRobot monitor keeps the API awake (watch the 750 free instance-hours/month).

8. Maintenance checklist

- Rotate the Neon database password and revoke unused GitHub tokens periodically.
- Watch the Render usage meter (free instance-hours) and the Brevo daily email count.
- For heavier traffic or to remove sleep entirely, move compute to a small VPS or Oracle Cloud (deploy guide included in the repo).

PDF Tool is live and production-grade: secure auth, durable data & files, automated CI, and email verification.